

Policy

Policy	State to track on paper
FIFO	arrival queue, oldest first
LRU / LRUv2	recency list, most recent first
MRU	the same recency list as LRU
LFU	each frame's count, plus a recency list per count

Test Scenarios

Notation:

L = on_load, **A** = on_access, **R** = on_remove, **P{...}** = pick_victim with those candidates, - means no victim.

Cases & Expected Values

#	Script	What it tests	FIFO	LRU / LRUv2	MRU	LFU
1	L0 L1 L2 · P{ }	empty candidate list	-	-	-	-
2	L0 L1 L2 · P{1}	single candidate	1	1	1	1
3	L0 L1 L2 · P{0,1,2}	arrivals only, no hits	0	0	2	0
4	L0 L1 L2 A0 · P{0,1,2}	a hit on the oldest frame	0	1	0	1
5	L0 L1 L2	Trace A: all, then 2	0, 0	2, 0	1, 1	2, 1

	A0 A0 A1 · P{0,1,2} P{0,1}	pinned				
6	L0 L1 L2 A0 A1 · P{0,1,2} P{0,1} P{1,0}	equal counts, tie-break	0, 0, 0	2, 0, 0	1, 1, 1	2, 0, 0
7	L0 L1 L2 L3 A2 · P{0,1,2, 3} P{1,2,3} P{2,3} P{3}	candidates shrink (pinning)	0, 1, 2 , 3	0, 1, 3, 3	2, 2, 2, 3	0, 1, 3, 3
8	L0 L1 L2 A0 A0 R0 L0 · P{0,1,2}	frame reuse	1	1	0	1
9	L0 L1 L2 L3 R1 · P{0,2,3} · R0 · P{2,3}	remove from the middle	0, 2	0, 2	3, 3	0, 2
10	L0 L1 L2 A1 A1 A1 A1 A2 · P{0,1,2} P{1,2}	one heavily used frame	0, 1	0, 1	2, 2	0, 2
11	L0 L1 L2 A0 A0 A0 A1 R2 · P{0,1} · R1 · P{0}	LFU minimum jumps	0, 0	0, 0	1, 0	1, 0
12	L0 L1 L2 A0 A0 · init · L2	init after activity	2	2	0	2

	$L_0 \cdot P\{0,2\}$					
13	$L_0 \ L_1 \ R_2$ $R_1 \ R_1 \cdot P\{0\}$	remove untracked, remove twice	0	0	0	0

Results

Methodology: Apply wrapper around each policy method in the **bufman::ReplacementPolicy** Load, access, remove and pick victim.

Build different policy objects from the classes each with its own underlying data structures, which allows us to call the functions and test the policy's methods. The test results were calculated manually and then compared to the policy's results.

1. empty candidate list

```
init(3) L0 L1 L2 | P{}
fifo : -
lru  : -
lruv2 : -
mru  : -
lfu  : -
```

2. single candidate

```
init(3) L0 L1 L2 | P{1}
fifo : 1
lru  : 1
lruv2 : 1
mru  : 1
lfu  : 1
```

3. arrivals only, no hits

```
init(3) L0 L1 L2 | P{0,1,2}
fifo : 0
lru  : 0
lruv2 : 0
mru  : 2
lfu  : 0
```

4. a hit on the oldest

init(3) L0 L1 L2 A0 | P{0,1,2}
fifo : 0
lru : 1
lruv2 : 1
mru : 0
lfu : 1

5. trace A from the guide (all, then frame 2 pinned)

init(4) L0 L1 L2 A0 A0 A1 | P{0,1,2} P{0,1}
fifo : 0 0
lru : 2 0
lruv2 : 2 0
mru : 1 1
lfu : 2 1

6. count tie (LFU tie-break)

init(3) L0 L1 L2 A0 A1 | P{0,1,2} P{0,1} P{1,0}
fifo : 0 0 0
lru : 2 0 0
lruv2 : 2 0 0
mru : 1 1 1
lfu : 2 0 0

7. pinned exclusion, shrinking candidates

init(4) L0 L1 L2 L3 A2 | P{0,1,2,3} P{1,2,3} P{2,3} P{3}
fifo : 0 1 2 3
lru : 0 1 3 3
lruv2 : 0 1 3 3
mru : 2 2 2 3
lfu : 0 1 3 3

8. frame reuse (remove then reload the same frame)

init(3) L0 L1 L2 A0 A0 R0 L0 | P{0,1,2}
fifo : 1
lru : 1
lruv2 : 1
mru : 0
lfu : 1

9. remove from the middle

init(4) L0 L1 L2 L3 R1 | P{0,2,3} ; then R0 | P{2,3}
fifo : 0 2
lru : 0 2
lruv2 : 0 2

mru : 3 3
lfu : 0 2

10. one hot frame

init(3) L0 L1 L2 A1 A1 A1 A1 A2 | P{0,1,2} P{1,2}
fifo : 0 1
lru : 0 1
lruv2 : 0 1
mru : 2 2
lfu : 0 2

11. LFU minimum jumps after a remove

init(3) L0 L1 L2 A0 A0 A0 A1 R2 | P{0,1} ; then R1 | P{0}
fifo : 0 0
lru : 0 0
lruv2 : 0 0
mru : 1 0
lfu : 1 0

12. init after activity starts clean

init(3) L0 L1 L2 A0 A0 | init(3) L2 L0 | P{0,2}
fifo : 2
lru : 2
lruv2 : 2
mru : 0
lfu : 2

13. robustness: remove untracked, remove twice

init(3) L0 L1 R2 R1 R1 | P{0}
fifo : 0
lru : 0
lruv2 : 0
mru : 0
lfu : 0